

Web Systems Application Development

Jason Kuruzovich

kuruzj@rpi.edu

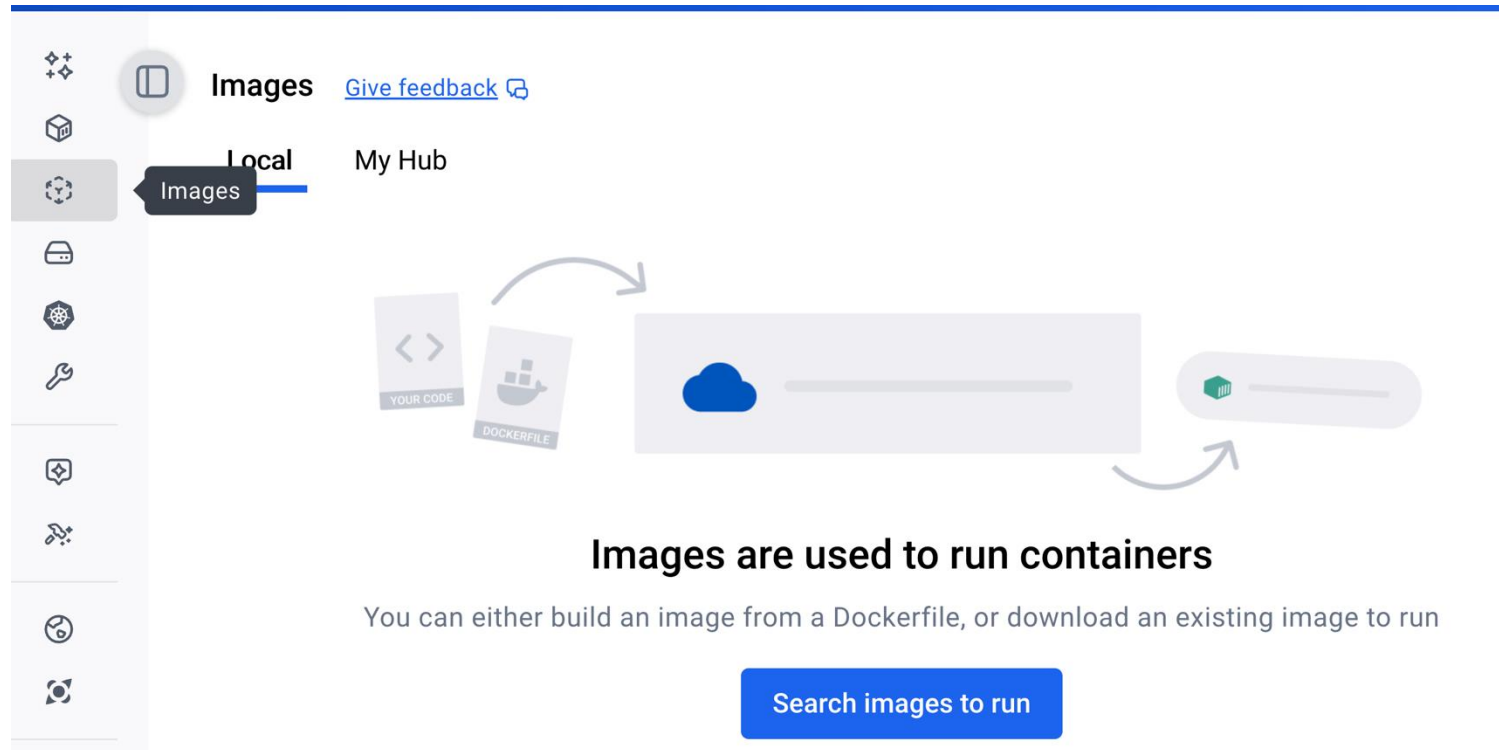


Agenda

- Troubleshooting – The Basics

Docker has key steps that need to be executed in order. What are they?

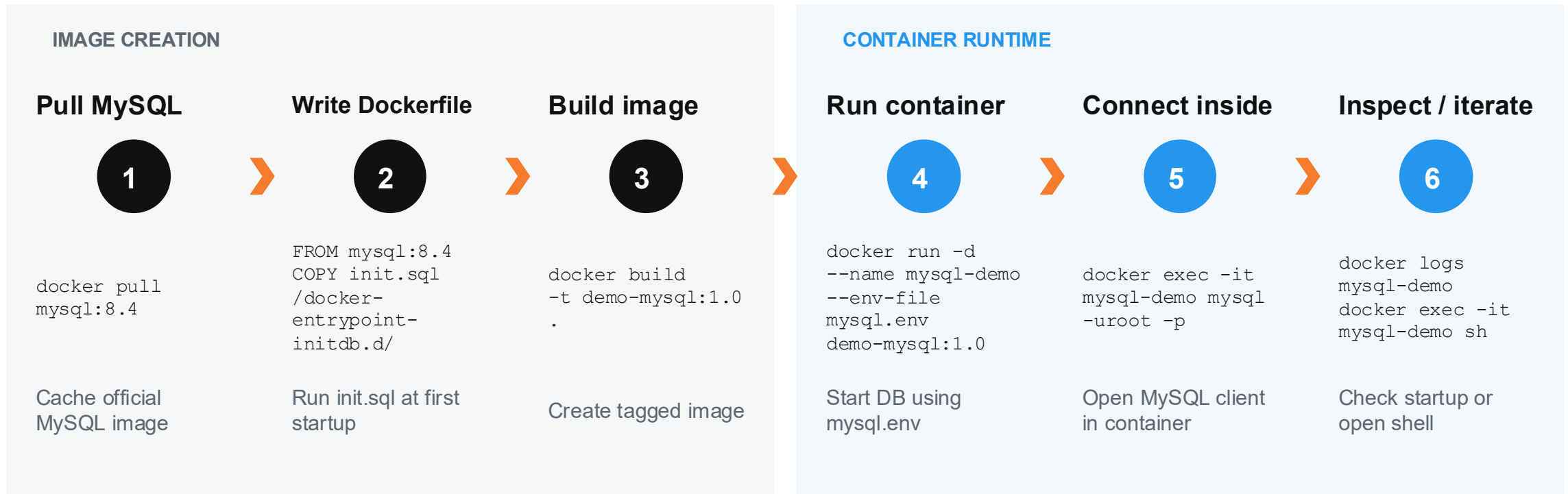
Finding Images



You can see existing images on disk and search images to run.

From the MySQL image to a running database

Customize MySQL, run it as a named container, and connect from inside the container.



FAST FEEDBACK LOOP

edit init.sql → docker build → docker rm -f mysql-demo → docker run ... → connect

Tip: docker exec runs the MySQL client inside the container, so host port 3306 does not need to be published.

Fix image-creation failures before trying to run

Pull → Dockerfile → build: each step must produce a valid input for the next.

1

Pull MySQL

Requires registry access and a valid image tag.

COMMON FAILURE

Timeout, denied access, or “manifest unknown.”

CHECK / FIX

```
docker pull mysql:8.4
docker image ls mysql
```

2

Write Dockerfile

Dockerfile and init.sql must be inside the build context.

CHECK / FIX

```
ls Dockerfile init.sql
docker build -f Dockerfile .
```

3

Build image

The build must finish and create the expected tag.

COMMON FAILURE

You aren't in the same directory as the dockerfile.

CHECK / FIX

```
Cd (target directory)
docker build --no-cache -t demo-mysql:1.0 .
docker image inspect demo-mysql:1.0
```

Trace runtime problems from container to connection

Image exists → container runs → MySQL becomes ready → the client can connect.

4

Run container

Requires the built image, a unique name, and available host ports.

COMMON FAILURE

Image not found, name already used, immediate exit, or port conflict.

CHECK / FIX

```
docker ps -a
docker logs mysql-demo
docker rm -f mysql-demo
# host access: -p 3307:3306
```

5

Connect to MySQL

The container must be running and MySQL must finish starting.

COMMON FAILURE

Connection refused, access denied, or connecting before MySQL is ready.

CHECK / FIX

```
docker ps
docker exec -it mysql-demo mysql -uroot -p
```

6

Inspect / iterate

Use container state and logs to identify the failed layer.

COMMON FAILURE

Bad environment values, initialization errors, or changes not rebuilt.

CHECK / FIX

```
docker logs --tail 50 mysql-demo
docker inspect mysql-demo
docker exec -it mysql-demo sh
```

DEPENDENCY CHAIN

IMAGE EXISTS → CONTAINER RUNNING → MYSQL READY → CONNECTION SUCCEEDS

Test the chain from left to right; fix the first broken condition.

Running Docker Build

3

Build image

The build must finish and create the expected tag.

COMMON FAILURE

You aren't in the same directory as the dockerfile.

CHECK / FIX

```
Cd (target directory)
docker build --no-cache -t demo-mysql:1.0 .
docker image inspect demo-mysql:1.0
```

- You must be in the same directory as the Docker file
- Find the repository on Finder (Mac) or Explorer (windows)
- Right click on the directory and then “Run in Terminal”

View container logs without opening a shell

Use recent, live, and time-filtered output to isolate startup and runtime errors.

1

Read recent output

Start with the last lines to find startup errors without flooding the terminal.

BEST FOR

A quick health and startup check.

COMMAND

```
docker logs --tail 100 mysql-demo
```

2

Follow live output

Stream new lines while reproducing the issue; press Ctrl+C to stop.

BEST FOR

Watching requests, retries, or crashes in real time.

COMMAND

```
docker logs --follow mysql-demo
```

3

Filter by time

Limit output to a recent window and add timestamps for correlation.

BEST FOR

Matching container events to an incident window.

COMMAND

```
docker logs --timestamps --since 10m mysql-demo
```

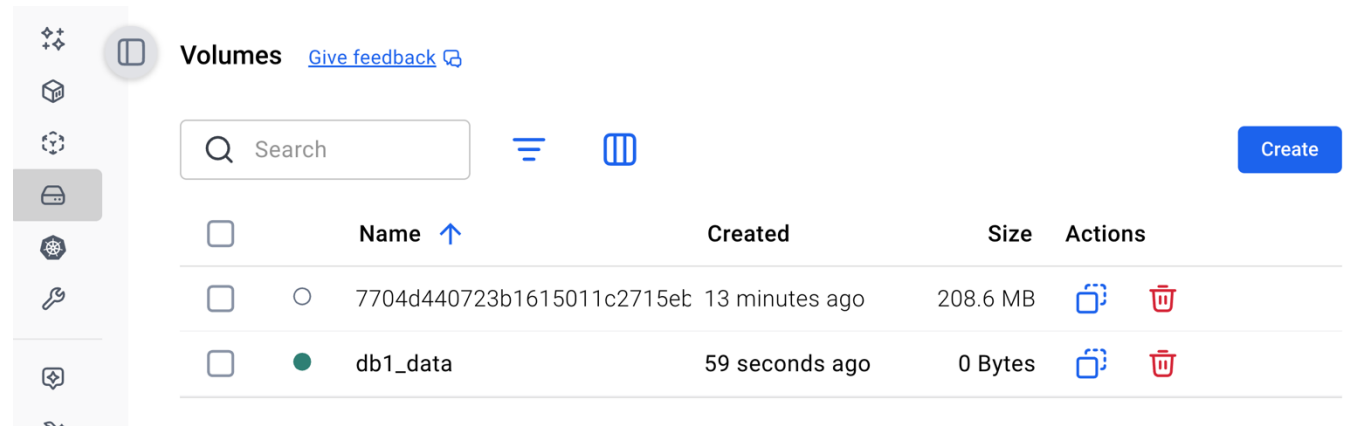
LOG DEBUG LOOP**RECENT → LIVE → TIME-FILTERED → INSPECT**

Tip: `docker logs` reads `stdout` and `stderr`; use `docker exec` only when the application writes to files.

Example 1

Key Learning Outcomes:

- Know how to run an image
- Connect to a container via bash
- Understand that Containers aren't operating systems
- Remove a container
- A container volume:
 - `docker rm -f db1`
 - `docker run --name db1 -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=app -p 3306:3306 -v db1_data:/var/lib/mysql -d mysql:8.4`



Example 2

Key Learning Outcomes:

- Our Dockerfile initiated copying of or SQL into the container. Database is seeded when it is run.
- If a container has no volume, it will run the initialization. If it does, it won't.
- *-v db2_data:/var/lib/mysql this is creating a volume but allowing Docker to select the location for the item.*

Adding an insert statement

```
docker rm -f db2
docker run --name db2 -e MYSQL_ROOT_PASSWORD=root -v db2_data:/var/lib/mysql -d itws-db:1
```



Choose volumes for managed data and bind mounts for host-controlled files

Both persist data outside the container. The key difference is whether Docker or the host controls the storage location.

	Named volume Docker-managed storage	Bind mount Host path mapped into a container
Location	Docker chooses and manages the location.	You specify an absolute or relative host path.
Lifecycle	Survives container removal until the volume is deleted.	The data is the host file or directory itself.
Portability	Easy to reuse across containers and Compose projects.	Depends on host paths, permissions, and filesystem.
Host visibility	Usually accessed through Docker rather than edited directly.	Immediately visible and editable with host tools.
Best use	Databases and application state.	Source code, configuration, and local development.
RUN SYNTAX	<code>-v db_data:/var/lib/mysql</code>	<code>-v "\$(pwd)/src:/var/www/html"</code>

Example 3 Differences Between Volume and Bind Mount

Learning Goals:

- Docker will choose where your MYSQL Volume is, typically stored within Docker system files.
- Bind Mount lists a specific volume for sharing.
 - Bind mounts are read/write by default.
 - Can make it read only
 - It is always in reference to the directory that you are in.

Example 4

- Our first running PHP Application
- More differences between bind mount

A — Run it from a bind mount

```
docker run -d --name gb -p 8080:80 -v "${PWD}/src:/var/www/html" php:8.3-apache
```



Open <http://localhost:8080>, sign the guestbook a couple of times, then look on your laptop:

```
cat src/data/visits.log
```



```
2026-09-01 01:15:00 from the bind mount
```



PHP created that directory and that file. The page header tells you who did it — `www-data`, the unprivileged user Apache runs as — and on your machine the file shows up owned by *you*. Docker Desktop translates ownership across the boundary so this just works. (On native Linux it does not, and matching user IDs is a genuine chore. Be glad.)

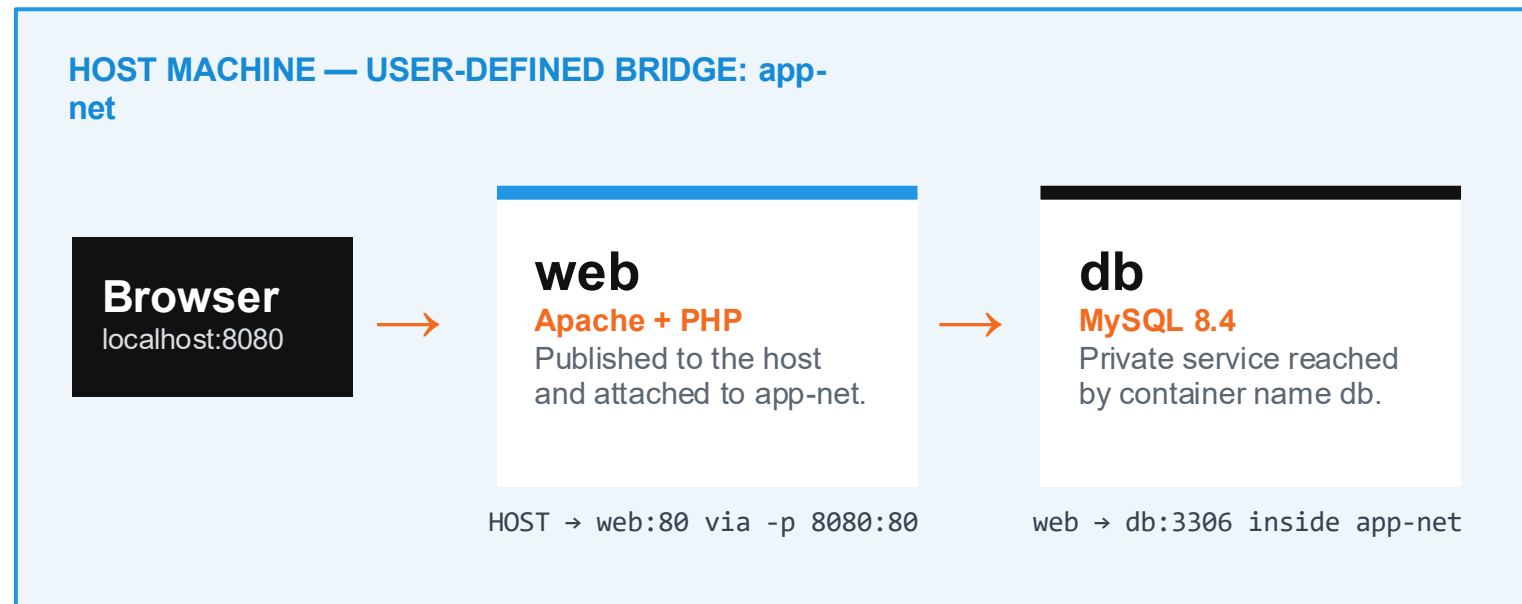
Delete `src/data/` when you want a clean slate. Note that you can do that from your own file manager — because the data is on your machine, not in the container.

B — Now take the mount away

Delete the `src/data/` folder, then:

User-defined networks let containers communicate by name

Place related containers on the same bridge network: Docker provides private connectivity and DNS-based discovery while published ports control host access.



Name-based discovery

Use db as the database hostname—not localhost.

Isolation by default

Only containers attached to the network can use its private path.

Publish selectively

Expose only services that the host or outside clients must reach.

CORE COMMANDS

```
docker network create app-net
docker run --network app-net --name db -d mysql:8.4
docker run --network app-net --name web -p 8080:80 -d my-php-app
docker network inspect app-net
```


Example 5

- Docker Networks enable connection between applications

We do this by hand first so that Compose, in Example 6, reads as *shorthand for things you already understand* rather than as magic.

```
example-5/  
├── db/  
│   ├── Dockerfile    # same as Example 2  
│   └── init.sql  
└── web/  
    ├── Dockerfile    # php:8.3-apache + pdo_mysql  
    └── src/index.php  # queries "db" by container name
```

A — Create a network

```
docker network create itwsnet  
docker network ls
```

A user-defined network gives you one thing a default bridge does not: **DNS by container name**. That is the whole trick in `index.php`, which connects to a host called `db`.

B — Build both images

```
docker build -t itws-db:1 ./db  
docker build -t itws-web:1 ./web
```

C — Run them, both on the network

```
docker run -d --name db --network itwsnet -e MYSQL_ROOT_PASSWORD=root itws-db:1  
docker run -d --name web --network itwsnet -p 8080:80 itws-web:1
```

DOCKER COMPOSE

Docker Compose turns a stack into one runnable application

Describe containers, networking, storage, and configuration once—then start the complete environment with one command.

WITHOUT
COMPOSE

Manual orchestration

Repeat long commands and coordinate networks, settings, and startup yourself.

```
docker run ...    docker network ...
```



WITH
COMPOSE

One Compose project

Capture services and relationships in compose.yaml, then launch them together.

```
docker compose up -d
```

CORE IDEA

ONE FILE → ONE PROJECT → REPEATABLE ENVIRONMENT

The file is declarative; the Compose CLI reconciles the running containers.

A Compose file defines services, storage, and connections

The current Compose Specification uses `compose.yaml`; a top-level `version:` field is not required.

```
services:
  web:
    build: ./php
    ports:
      - "8080:80"
    volumes:
      - ./src:/var/www/html
  db:
    image: mysql:8.4
    environment:
      MYSQL_DATABASE: app

volumes:
  db_data:
```

1

Services

Each service becomes one or more containers, such as `web` and `db`.

2

Configuration

Images, builds, ports, environment variables, and startup behavior live together.

3

Networks

Services join a project network and reach each other by service name.

4

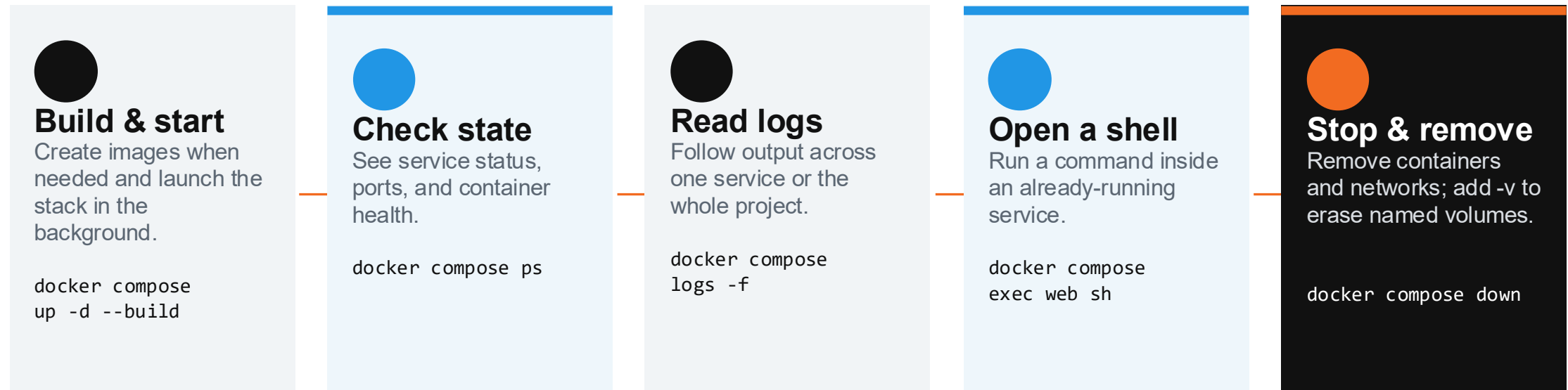
Volumes

Named volumes preserve database state; bind mounts expose local source code.

COMPOSE WORKFLOW

The Compose CLI manages the whole application lifecycle

Commands operate on the project, so related services start, stop, and report status together.



DAILY LOOP

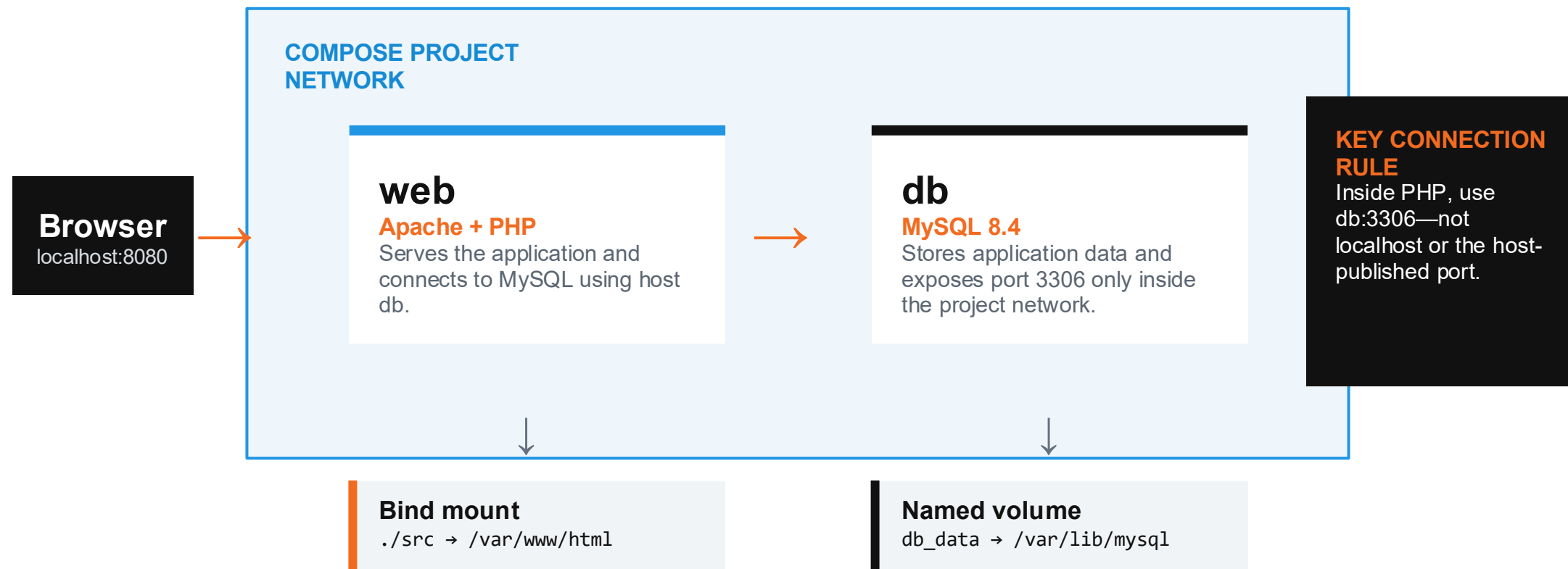
UP → PS → LOGS / EXEC → DOWN

Use “`docker compose down -v`” only when you intentionally want a fresh database.

LAMP WITH COMPOSE

In a LAMP stack, Compose makes service boundaries explicit

Apache and PHP share one container; MySQL runs separately and is discovered through the service name db.



The LAMP Compose file encodes the complete local environment

Build PHP with database extensions, then let Compose coordinate the web and database services.

compose.yaml — WEB SERVICE

```
services:
  web:
    build: ./php
    ports:
      - "8080:80"
    volumes:
      - ./src:/var/www/html
    environment:
      DB_HOST: db
      DB_NAME: app
      DB_USER: app
      DB_PASSWORD: apppass
    depends_on:
      db:
        condition: service_healthy
```

compose.yaml — DATABASE + STORAGE

```
db:
  image: mysql:8.4
  environment:
    MYSQL_ROOT_PASSWORD: root
    MYSQL_DATABASE: app
    MYSQL_USER: app
    MYSQL_PASSWORD: apppass
  volumes:
    - db_data:/var/lib/mysql
  healthcheck:
    test: ["CMD-SHELL",
           "mysqladmin ping -h localhost -proot"]
    interval: 5s
    timeout: 3s
    retries: 10

volumes:
  db_data:
```

RUN AND VERIFY

Bring up the LAMP stack, verify each layer, then clean up deliberately

Test from infrastructure to application behavior; fix the first layer that fails.

1	Start the project Build PHP and create the network, services, and database volume.	EXPECTED SIGNAL Both services reach running / healthy state.	COMMAND <code>docker compose up -d --build</code>
2	Inspect status Confirm port 8080 and MySQL health before testing the page.	EXPECTED SIGNAL web is Up; db is Up (healthy).	COMMAND <code>docker compose ps</code>
3	Test the path Open the page, then follow both services if the request fails.	EXPECTED SIGNAL Browser reaches PHP; PHP connects to host db.	COMMAND <code>open http://localhost:8080</code> <code>docker compose logs -f web db</code>
4	Reset intentionally Normal cleanup preserves MySQL data; a volume reset removes it.	CHOOSE THE OUTCOME Keep data with down; start fresh with down -v.	COMMAND <code>docker compose down</code> <code>docker compose down -v</code>

DEBUG ORDER

CONFIG → CONTAINERS → HEALTH → NETWORK → APPLICATION

COMPOSE WATCH

Compose Watch turns file changes into automatic container updates

Use it during development to get fast feedback without manually rebuilding and restarting the stack after every edit.

1	Sync source files Best for PHP, templates, CSS, and other files read immediately.	WATCH RESPONSE Copies changed files into the running container.	EXAMPLE sync: ./src → /var/www/html
2	Sync + restart Best for configuration loaded only when the process starts.	WATCH RESPONSE Copies the change, then restarts the service.	EXAMPLE sync+restart: ./config → /app/config
3	Rebuild the image Best for Dockerfile or dependency-file changes.	WATCH RESPONSE Rebuilds the image and recreates the service.	EXAMPLE rebuild: ./php/Dockerfile
4	Run the dev loop Start Watch once, then continue editing locally.	WHY IT HELPS No repeated build, remove, and run commands.	COMMAND Docker compose watch. (or) docker compose up --watch

DEVELOPMENT
VALUE

FASTER FEEDBACK → SAME CONTAINER ENVIRONMENT → LESS MANUAL WORK

Example 6

- Shows how to use a non-lamp docker compose file that has

Example 6 — The same thing, declared once

Example 5 took six commands in an exact order, and it still had a startup race. Here it is one file and one command — against a **different stack**, so that what you learn is about Compose and not about PHP.

```
example-6/
├── docker-compose.yml
└── simple-server/
    ├── Dockerfile
    ├── server.js      # Express + Mongoose: a visitors collection
    └── package.json
```

Node and Express instead of Apache and PHP. MongoDB instead of MySQL. Every idea from Example 5 carries over unchanged.

Treat Compose environment variables as configuration, not secret storage

Separate values used to render the Compose model from values passed into containers, then make every source and fallback deliberate.

compose.yaml — EXPLICIT AND VALIDATED

```
services:
  web:
    image: "myapp:${APP_TAG:-dev}"
    env_file:
      - ./config/web.env
    environment:
      APP_ENV: ${APP_ENV:-development}
      DB_HOST: ${DB_HOST:?set DB_HOST}
    secrets:
      - db_password

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

1 Separate the two jobs

`${...}` interpolates the Compose file; `environment` and `env_file` configure the container.

2 Make defaults and requirements visible

Use `:-` for safe defaults and `?:` to fail early when a required value is missing.

3 Use environment-specific files deliberately

Keep non-secret values in named files and select them explicitly with `--env-file`.

4 Keep secrets out of environment variables

Use Compose secrets for passwords, tokens, and certificates instead of plain-text variables.

5 Inspect the resolved configuration

Check interpolation and precedence before startup; do not debug a value only from inside the container.

VALIDATE BEFORE RUNNING

```
docker compose --env-file .env.dev config • docker compose config --environment
```

Homework 1

- Create MYSQL DOCKER FILE.
- Create Apache Dockerfile.
- Create Docker Compose
- Answer the writeup questions

Next week Testing!

1. Mauro Chojrin, *Modern Testing with PHP* — chapters 1, 2, 7

[Read it through the RPI library](#) — sign in with your RPI credentials when prompted. If you are off campus and it will not load, go to the library site first, then follow the link again.

Chapter		
1	About Automated Tests	what a test is for, and why "it works when I click it" does not count
2	Introduction to PHPUnit	your first real test file: arrange, act, assert, and reading a failure
3	Case Study	Read about the case study. We will use it in the homework.

2. Playwright — *Writing tests*, 15 minutes

playwright.dev/docs/writing-tests

Playwright drives a real browser from a script and asserts what a user would see. It is language-agnostic — it does not know or care that the server is PHP. Read the page; do not install anything — on Tuesday it runs inside a container. You will write Playwright tests yourself in Homework 2.

Optional

3. Ian Cooper, *TDD, Where Did It All Go Wrong* — 1 hour, video

youtube.com/watch?v=EZ05e7EMOLM (DevTernity 2017)

Argues that most people test the wrong unit and end up with suites that block every refactor. This will change how you test, and it is the talk I will be arguing with in class. Watch it at 1.5x if you must, but watch all of it.

<https://github.com/RPI-WS-fall-2026/itws2110-course/blob/main/weeks/week-03.md>